Lesson 13 · Working in Visual Studio

A companion for teams that live in the IDE — Community, Professional and Enterprise compared without prices, the files Visual Studio reads (.vsconfig, .slnx, launchSettings, .runsettings, .editorconfig), the debugger and Test Explorer, and where VS Code and Rider fit

▤ Contents

1 · Why an IDE, when the CLI builds everything 1
- 1.1 · The numbers to leave this lesson with 2
- 1.2 · Two front ends, one set of files 2
- 1.3 · Where the IDE wins, where the CLI wins 3
- 1.4 · Concept map — IntelliJ, Android Studio and VS Code habits to Visual Studio 3

2 · The editions — Community, Professional, Enterprise 3
- 2.1 · Which edition — the decision as a flow 4
- 2.2 · One card per edition — and what Professional buys a team 4
- 2.3 · What each edition has — the Enterprise-only rows 5
- 2.4 · The 14, by section of the page 5

3 · Install, workloads and .vsconfig 5

4 · Solutions and projects in the IDE 6
- 4.3 · One job, three places — Solution Explorer, the NuGet manager, the CLI 7

5 · The inner loop: debugging 7
- 5.2 · What one press of F5 does 8
- 5.3 · The breakpoint kinds, what each is for, and where this lesson's code uses it 9
- 5.7 · A window or a setting for each question 10
- 5.8 · Hot Reload and Edit and Continue — the limits Microsoft documents 11
- 5.9 · Attaching to a process you did not start 11

6 · Testing in the IDE 11
- 6.5 · Test Explorer, the CLI and lesson 10's gates — the same tests, three doors 13

7 · Code quality and AI assistance in the IDE 13
- 7.2 · Where a style setting can live, and who sees it 14

8 · Performance and diagnostics 15
- 8.4 · Bytes the report allocates above pricing alone (Release) 16
- 8.5 · Calls to Price in one measured pass 16

9 · Legacy estates in the IDE 16

10 · Other editors — VS Code with the C# Dev Kit, and Rider 17
- 10.1 · A decision table — vendor facts verified, the last column is an ESTIMATE 17

11 · Hands-on — a guided debug session over a correct program 17
- 11.1 · Five questions, the IDE feature that answers each, and the answer 17
- 11.3 · The files Visual Studio reads in this lesson, and who else reads them 18
- 11.4 · Every file that configures the IDE or the build, by size 19
- 11.5 · What to carry over, learn fresh and unlearn 19

Page numbers are resolved from the rendered PDF. Code panels (numbered N.M in the text) are not listed here; they are cut from the compiled samples; every claim about Visual Studio was read on an official page on 2026-09-21 and is linked beside it.

1 · Why an IDE, when the CLI builds everything

The first twelve lessons drive .NET from a terminal because a terminal is what CI runs — but your team probably opens Visual Studio every morning. A JVM developer arrives with IntelliJ IDEA or Android Studio in the muscle memory: a run configuration, inspections that travel with the repository, a debugger that evaluates expressions. Visual Studio has an answer for each, and most of them live in *files you can commit*. This lesson is a companion, not a prerequisite: read it after lesson 07 (Solutions, MSBuild & Mono-repos) (solutions and MSBuild), or the day you join a team that lives in the IDE. It assumes lesson 01 (The .NET Platform Map) for the SDK and CLI.

One rule keeps the IDE and the command line from drifting apart: the solution and project files are the source of truth, and both front ends only read them. Build > Build Solution and `dotnet build` run the same MSBuild targets over the same `.csproj`, `Directory.Build.props` and `global.json`. If a build passes in one and fails in the other, something outside the files both front ends read differs — an installed workload, an SDK, or a file only the command line reads: `Directory.Build.rsp` (a switch the IDE never sees) and `Directory.Solution.props` / `.targets` (they run only when

MSBuild builds the solution, never inside Visual Studio). See lesson 07 (Solutions, MSBuild & Mono-repos) §4.5 and MSBuild response files. §3 shows how to make that list shorter.

So the IDE is not where your build lives; it is the best instrument for looking at a running program. Its strengths are the ones a terminal cannot match — a debugger you can aim, a test runner that shows state, a profiler, designers, and refactorings that see the whole solution. Its weakness is the one you already know: a setting that lives only in a per-user Options dialog is invisible to a colleague and to CI. Every section from §3 to §7 is therefore anchored on a file that carries the setting into the repository. Facts about the product are **VERIFIED** against an official page read on 2026-09-21; the samples prove only the parts that are code and files.

1.1 · The numbers to leave this lesson with

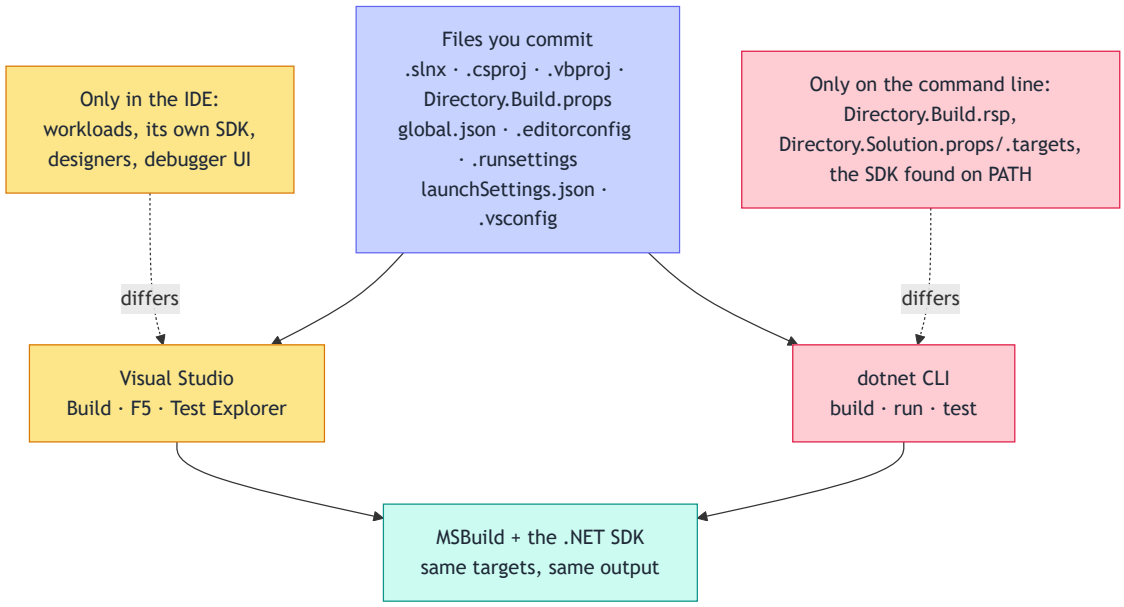
VISUAL STUDIO 2026	FIRST 2026 RELEASE	SAMPLE PROJECTS	TESTS IN TEST EXPLORER	FILES THE IDE READS
18.10.1	**18.0.0**	**4 projects**	**18 tests**	**12 files**
current Stable release, 2026-09-15 · VERIFIED 2026-09-21	released 2025-11-11 · release history	C# and VB · build without Visual Studio · measured	14 Tariff · 3 Batch · 1 Environment · measured	8 settings files + 4 project files · measured

KEY: • Same idea • Renamed • Different • Trap • Verified • Estimate • Measured tooling

i Before you start

1. **You need lesson 01 (The .NET Platform Map) and lesson 07 (Solutions, MSBuild & Mono-repos).** Lesson 01 gave you the CLI and the `global.json` trap; lesson 07 gave you solutions, `Directory.Build.props` and analyzers. Nothing here replaces them — it shows the IDE side of the same files.
2. **You do not need Visual Studio to run anything in this lesson.** 4 sample projects (C#, Visual Basic, xUnit v3) build and pass on the pinned SDK with `python 0-script/verify_samples.py --only 13`. Open `samples/L13.slnx` in Visual Studio if you have it, and every step in §11 works there too.
3. **Every claim about the IDE carries a source.** Editions, licence terms, version numbers and feature gating change several times a year, so this lesson prints no prices and labels anything it could not confirm on an official page. Read the page linked next to a claim before you act on it.

1.2 · Two front ends, one set of files



indigo = files you commit · teal = shared build engine · amber = only the IDE · rose = only the command line — a difference comes from an amber or rose box, never from the files both front ends read

1.3 · Where the IDE wins, where the CLI wins

Job	Reach for the IDE when	Reach for the CLI when	Source
Debugging	you need to aim: conditional and dependent breakpoints, data tips, Just My Code, attach to a process	you only need a stack trace in a log	Breakpoint docs
Testing	you want state, grouping by trait and a one-click debug of one test	the same tests must run identically in CI: `dotnet test`	Test Explorer docs
Refactoring	the change spans a solution — Quick Actions and Refactorings	you script a bulk style fix: `dotnet format`, as in lesson 07 (Solutions, MSBuild & Mono-repos)	Code style docs
Designers	you edit a WinForms form for .NET — the designer runs beside the IDE	you only build or test the project: the form is code, and MSBuild compiles it	WinForms designer
Reproducible builds	never the source of truth — it reads the same files	always: CI, release, anything another machine must repeat	global.json docs
Profiling	you want a guided tool for a Release build	the machine has no Visual Studio: `dotnet-trace`, `dotnet-counters`	Profiling docs

1.4 · Concept map — IntelliJ, Android Studio and VS Code habits to Visual Studio

You already know	In .NET	Kind	Watch for
A Run/Debug Configuration	a profile in `launchSettings.json`	• Renamed	A file in the repository, and `dotnet run -lp` reads it too (5.1)
Gradle `tasks.test { environment(…) }`	`.runsettings` `EnvironmentVariables`	• Renamed	Autodetected only under that exact name at the solution root (6.1)
An IDE code-style scheme and inspection profile	`.editorconfig` severities and analyzers	• Different	Tools > Options belongs to your account and is not enforced at build (7.2)
A breakpoint condition, "Evaluate and log"	a conditional breakpoint, a tracepoint	• Same idea	Conditions, hit count, filter, dependent, temporary — all in one dialog (5.3)
`toString()` shown in the debugger	`[DebuggerDisplay]`, `DebuggerTypeProxy`	• Renamed	Attributes in code; DebuggerDisplay wins over an overridden `ToString()` (5.5)
HotSwap in a debug session	Hot Reload and Edit and Continue	• Different	A documented list of edits and build modes that do not apply (5.8)
A shared plugin list	`.vsconfig`: workloads, components, extensions	• Different	It installs components of the IDE itself, not only plugins (3.1)
Attach to a remote JVM	Attach to Process — `w3wp.exe`, `dotnet.exe`	• Same idea	Source and symbols must match the running binary (5.9)
Run with Coverage	Analyze Code Coverage for All Tests	• Trap	Counts blocks, not lines; its edition gating changed between releases (2.3, 6.5)

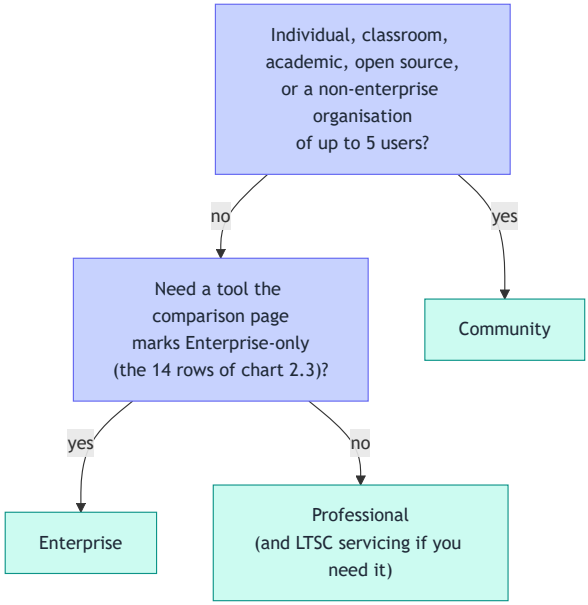
2 · The editions — Community, Professional, Enterprise

Three editions share one IDE; licence terms separate them first and the Enterprise-only tools second. Community is the full IDE for a stated list of users: individual developers, classroom learning, academic research, contributing to open source, and non-enterprise organisations with up to five users. The page also defines the organisations that are *excluded* — more than 250 PCs or more than US$1 million in annual revenue — so a commercial team of any size has to read that definition before it installs (Visual Studio editions comparison, read 2026-09-21). When Community's rules exclude an organisation, the choice is Professional or Enterprise.

Professional is what an organisation outside Community's rules assigns to developers who use none of the Enterprise-only tools — the comparison page marks 14 rows that way (2.3). Servicing is the clearest difference you will feel: Microsoft ships Visual Studio 2026 on a modern lifecycle of two years of servicing per annual release, the second year security-only, delivered through the LTSC channel; the current Stable release is 18.10.1 (2026-09-15) and 18.0.0 shipped on 2025-11-11 (Visual Studio 2026 release history · Visual Studio 2026 announcement). Community follows the Stable and Insiders channels and must keep up within 180 days; Professional and Enterprise can also hold an LTSC baseline (Visual Studio product lifecycle and servicing).

Feature gating moves, so read the feature's own page for your version. Code coverage is the example that will bite you: the Microsoft page says it is available in Enterprise, Community and Professional in Visual Studio 2026, and that in Visual Studio 2022 and earlier it was limited to Enterprise (code coverage in Visual Studio), while the Test Explorer page still says Enterprise in one sentence (run unit tests with Test Explorer). When Microsoft's pages disagree, trust the menu in your own edition and record which page you followed. **No prices appear in this lesson**: they and the subscription terms change, and only Microsoft's page is authoritative.

2.1 · Which edition — the decision as a flow



indigo = a question · teal = an outcome · the licence terms Microsoft publishes bind you, not this chart — confirm there before you install

2.2 · One card per edition — and what Professional buys a team

verified

1 Community `tooling` • Verified

The full IDE, free, for a stated list of users — the comparison page names them.

WHO	Individuals, classroom learning, academic research, open-source work, and non-enterprise organisations with up to 5 users
EXCLUDED	An enterprise organisation: more than 250 PCs or more than US$1 million in annual revenue (as the page defines it)
CHANNELS	Stable and Insiders only; must be updated within 180 days
READ	Visual Studio editions comparison · Visual Studio product lifecycle and servicing

2 Professional `tooling` • Verified

The paid team edition: what an organisation outside Community's rules assigns to developers.

BUYS	The same IDE, licensed for a team, plus the LTSC servicing channels Community does not get
TESTING	Test Explorer and, in VS 2026, code coverage; not Live Unit Testing
COPILOT	Sold by GitHub plan, not by edition — see §7
READ	Visual Studio product lifecycle and servicing · code coverage in Visual Studio

3 Enterprise `tooling` • Verified

Professional plus the 14 rows the comparison page marks Enterprise-only (chart 2.3).

ADDS	Live Unit Testing, Microsoft Fakes, IntelliTest, IntelliTrace, .NET memory dump analysis, Snapshot Debugger, Time Travel Debugging (Preview), architecture and layer-diagram tools, Tier Interaction Profiling, and more
ASK	Does your team use any of them? Read the whole list on the comparison page before you buy
READ	Visual Studio editions comparison · Live Unit Testing · Microsoft Fakes
NOTE	Gating moves between releases (see 2.3) — check the page for your version

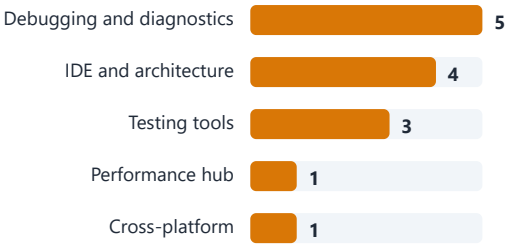
2.3 · What each edition has — the Enterprise-only rows



Com = Community · Pro = Professional · Ent = Enterprise · ✓ = the comparison page marks it “feature available” · — = “feature not available” · rubric, not a benchmark · read 2026-09-21

14 rows are Enterprise-only; code coverage, last row, is in all three. The cells on Visual Studio editions comparison are icons whose accessible labels this chart reads; Tier Interaction Profiling is the one row stated as text. • Verified .

2.4 · The 14, by section of the page



count of rows per section of the comparison page · same read as 2.3

Debugging and testing tools carry most of them. Coverage moved from Enterprise-only (Visual Studio 2022 and earlier) to all three in 2026 (code coverage in Visual Studio). Gating moves; read the page for your version.

3 · Install, workloads and .vsconfig

Visual Studio installs by workload, and a `.vsconfig` file makes a team's list of workloads a thing you commit instead of a wiki page. The Installer groups components into workloads — *.NET desktop development* and *ASP.NET and web development* are the two this track needs — and lets you tick individual components on top. You export the selection to a `.vsconfig` JSON file from the Installer (More > Export configuration), commit it, and from then on opening a solution whose root holds that file makes Visual Studio detect the missing components and offer to install them. The same file drives an unattended install: the edition bootstrapper with `--config` on a fresh machine, `setup.exe modify --config` on an existing one — `modify`, not `update`, which only refreshes components already installed (import or export installation configurations).

The file has two arrays: `components` is required and may be empty, `extensions` is optional. An extension must be a `.vsix`; only extensions hosted on the Visual Studio Marketplace trigger the automatic missing-component prompt, and a silent install needs `--allowUnsignedExtensions` for unsigned ones. The component IDs come from Microsoft's list (workload and component IDs), which is labelled Visual Studio 2022 — export from your own installer to see the exact IDs it writes.

Revisit lesson 01's SDK trap from this side: the IDE installs its own .NET SDK, and `global.json` is read from two different places. Visual Studio installs a single copy of the SDK and, when you upgrade Visual Studio, removes the SDK version it installed before — even a different major — unless you install the stand-alone SDK, in which case Visual Studio no longer updates it. The `dotnet` muxer looks for `global.json` upward from the *current directory*; MSBuild's SDK resolver looks upward from the folder with the *solution file* (global.json and rollForward). So when a build passes in the IDE and `dotnet build` says “No .NET SDKs were found”, start where lesson 01 (The .NET Platform Map) does — the PATH — before you suspect the solution.

3.1 · .vsconfig — the workloads this track needs

JSON — samples/.vsconfig

```
{
  "version": "1.0",
  "components": [
    "Microsoft.VisualStudio.Workload.ManagedDesktop",
    "Microsoft.VisualStudio.Workload.NetWeb",
    "Microsoft.NetCore.Component.SDK",
    "Microsoft.NetCore.Component.DevelopmentTools",
    "Microsoft.VisualStudio.Component.Roslyn.LanguageServices",
    "Microsoft.VisualStudio.Component.NuGet",
    "Microsoft.VisualStudio.Component.DiagnosticTools",
    "Microsoft.VisualStudio.Component.IISExpress",
    "Microsoft.VisualStudio.Component.Git"
  ]
}
```

Nine IDs, one file, beside the solution. Two workloads, the .NET SDK and its development tools, the C# and Visual Basic language services, NuGet, the diagnostic tools, IIS Express and Git. It contains no `extensions` array because nothing here needs one. Commit it at the folder that holds `L13.slnx`: that is the "solution root" Visual Studio checks (import or export installation configurations).

4 · Solutions and projects in the IDE

Solution Explorer is a view of the solution file, and the modern solution file is a dozen lines of XML you can read. Visual Studio reads both `.sln` and `.slnx` from 17.14 (the simpler solution file format), and since .NET 10 `dotnet new sln` writes `.slnx` by default (dotnet new sln defaults to SLNX); lesson 07 (Solutions, MSBuild & Mono-repos) measures the difference. The Solution Items folder in `L13.slnx` below is a display grouping only.

A big solution opens faster with a solution filter, which is a committed list of projects to load. Choose Do not load projects, load the ones you need, and File > Save Solution As Filter writes a `.slnf`; from a shell, `devenv /donotloadprojects MySln.sln` does the same. Right-click a project for Load Direct Dependencies or Load Entire Dependency Tree; a project added to the solution later appears as unloaded (solution filters in Visual Studio). It is an IDE convenience, not a build boundary: MSBuild still follows references, as lesson 07 (Solutions, MSBuild & Mono-repos) shows.

C# and Visual Basic projects sit in one solution, share one build and one debugger. The attribute that shapes how the debugger shows a value is the same in both languages; the type around it is not. Visual Basic has no `record struct`, so the same idea is a structure with two read-only properties and a constructor — the compare below is the whole difference.

4.1 · The same debugger attribute on a C# record struct and a VB structure

C# — Domain.cs · #region MoneyDisplay

```
// {Amount} and {Currency,nq} are expressions the debugger
// evaluates: Locals, Watch and data tips show the result.
[DebuggerDisplay("{Amount} {Currency,nq}")]
public readonly record struct Money(
    decimal Amount, string Currency)
{
    public static Money Thb(decimal amount) =>
        new(amount, "THB");
}
```

Visual Basic — Domain.vb · #region MoneyDisplay

```
' The attribute is identical to C#'s. The type around it is
' longer: Visual Basic has no record struct.
<DebuggerDisplay("{Amount} {Currency,nq}")>
Public Structure Money
    Public ReadOnly Property Amount As Decimal
    Public ReadOnly Property Currency As String

    Public Sub New(amount As Decimal, currency As String)
        Me.Amount = amount
        Me.Currency = currency
    End Sub

    Public Shared Function Thb(amount As Decimal) As Money
        Return New Money(amount, "THB")
    End Function
End Structure
```

One attribute, two type declarations. `{Amount} {Currency,nq}` is evaluated by the debugger; `,nq` removes the quotes around a string. Left: a `readonly record struct` in six lines. Right: Visual Basic writes the properties and the constructor out.

4.2 · L13.slnx — the whole solution file

MSBuild / XML — samples/L13.slnx

```
<Solution>
  <Folder Name="/Solution Items/">
    <File Path=".editorconfig" />
    <File Path=".runsettings" />
    <File Path=".vsconfig" />
  </Folder>
  <Folder Name="/src/">
    <Project Path="L13.DebugLab/L13.DebugLab.csproj" />
    <Project Path="L13.DebugLabVb/L13.DebugLabVb.vbproj" />
    <Project Path="L13.PerfLab/L13.PerfLab.csproj" />
  </Folder>
  <Folder Name="/tests/">
    <Project Path="L13.DebugLab.Tests/L13.DebugLab.Tests.csproj" />
  </Folder>
</Solution>
```

Solution folders and projects, nothing else. `/Solution Items/` lists the three files Visual Studio and the test runner read; `/src/` and `/tests/` group the four projects, C# and Visual Basic side by side. There is no configuration matrix and no GUID — the build order comes from each project's own references.

4.3 · One job, three places — Solution Explorer, the NuGet manager, the CLI

Job	In Visual Studio	In a file or on the command line	Source
Open code	File > Open > Folder needs no solution or project file; a folder with a `.sln` must be opened as a solution; custom build tasks and launch parameters come from JSON files	`dotnet build` takes a project, a solution or a file-based app	Open Folder docs · dotnet build docs
Add a package	Project > Manage NuGet Packages: Browse, Installed and Updates tabs; .NET SDK-style projects require `PackageReference`	`dotnet package add <name>` adds or updates a `PackageReference` in the project file (.NET 10 form; earlier SDKs: `dotnet add package`)	NuGet in VS docs · dotnet package add docs
One version list	nothing IDE-specific: it applies to all `PackageReference`-based MSBuild projects	`Directory.Packages.props` with `ManagePackageVersionsCentrally` and `PackageVersion` items; projects then omit `Version`, as in lesson 07 (Solutions, MSBuild & Mono-repos)	Central Package Mgmt docs

5 · The inner loop: debugging

F5 is three steps — build, start with a launch profile, attach the debugger — and only the second one is yours to configure. The profiles live in `Properties/launchSettings.json`: the toolbar's run dropdown lists them, each with a `commandName` (`Project` for a normal process), arguments and environment variables. The file is used only on the local development machine and is not deployed when the app is published (launchSettings.json in ASP.NET Core). IntelliJ's run configurations are the closest thing you know, with one difference that matters here: this one is a plain file in the repository, and the CLI reads it too — `dotnet run -lp "Show quote 137"` starts the profile below.

A breakpoint is a condition, and the conditions are the whole skill. Right-click the red dot and choose Conditions: a conditional expression (break when it is true, or when its value changes), a hit count, or a filter on machine, process or thread. Add an action and it becomes a tracepoint — it writes to the Output window and does not stop. A dependent breakpoint stays disabled until another one has been hit (use the right type of breakpoint). The sample's batch loop is built for this: 240 quotes, four of which are declined, and one loop body worth stopping in once.

Then shape what the debugger shows. `[DebuggerDisplay]` puts a one-line summary in the Locals window and in data tips; a `DebuggerTypeProxy` replaces the raw fields with a view you design; `[DebuggerStepThrough]` tells F11 to skip plumbing. None of them changes what the program does.

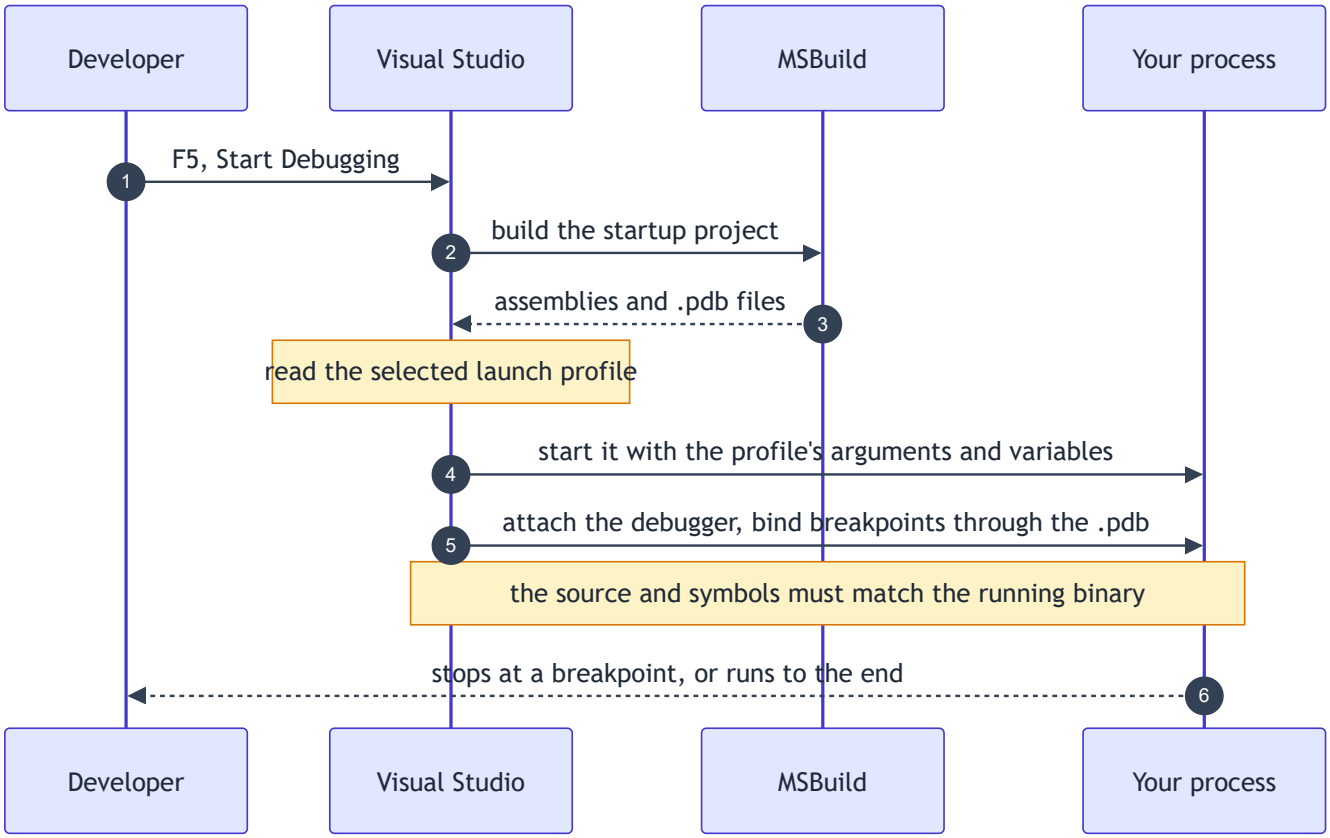
5.1 · launchSettings.json — three profiles for one project

JSON — samples/L13.DebugLab/Properties/launchSettings.json

```json
{
  "$schema": "https://json.schemastore.org/launchsettings.json",
  "profiles": {
    "Batch": {
      "commandName": "Project",
      "commandLineArgs": ""
    },
    "Show quote 137": {
      "commandName": "Project",
      "commandLineArgs": "--show 137"
    },
    "Show quote 47 (declined)": {
      "commandName": "Project",
      "commandLineArgs": "--show 47",
      "environmentVariables": {
        "DOTNET_ENVIRONMENT": "Development"
      }
    }
  }
}
```

Names are the interface. The dropdown shows `Batch`, `Show quote 137` and `Show quote 47 (declined)`; `dotnet run -lp` takes the same names. The third profile shows where an environment variable goes; this program does not read it, so it changes nothing here. `dotnet run` without `-lp` uses the first profile, in file order, whose `commandName` it supports; a Visual Basic project keeps the file at `My Project/launchSettings.json` instead (dotnet run). The two Show profiles start `Program.Show`, which prices one quote and returns before the batch loop — a breakpoint in `QuoteBatch.Run` hits only under `Batch`.

5.2 · What one press of F5 does



solid arrows = requests · dashed = results · steps 1–3 are the MSBuild step the CLI shares, dotnet run -lp reads the same profile · steps 5–6 exist only because a debugger is attached

5.3 · The breakpoint kinds, what each is for, and where this lesson's code uses it

Kind	How you set it	Use it when	In the samples
Conditional	right-click the breakpoint > Conditions > Conditional Expression (*Is true* or *When changed*); a + marks it	one iteration of a loop is wrong	`i == 137` in `QuoteBatch.Run`
Hit count	Conditions > Hit Count	the loop misbehaves after N passes	138, the same quote (`i` starts at 0)
Filter	Conditions > Filter: machine, process id or name, thread id or name	only one process or thread matters	not needed — the samples are single-threaded
Tracepoint	Actions: log a message to the Output window, do not stop	you want a trace without editing code	log `{i}` and `{request.Coverage}` on the marked line of 5.4
Dependent	Only enable when the following breakpoint is hit	a common path matters only from one caller	a stop in `Request` that waits for the loop stop
Temporary	Disable breakpoint once hit (VS 2026 wording)	you want to stop once, then never again	the throw in `ClaimsLoading`
Data	Autos, Watch or Locals > right-click > Break when value changes	you need to know who changed a property	**not** on `BatchResult` — a type with a `DebuggerTypeProxy` cannot host one

Source: use the right type of breakpoint. The page also lists what a data breakpoint cannot watch — static variables, classes with the `DebuggerTypeProxy` attribute and fields inside structs — and the hardware limit of 4 on x64 and x86. In Visual Studio 2022 the temporary breakpoint reads “Remove breakpoint once hit”. ● Verified 2026-09-21.

5.4 · A loop worth stopping in — 240 quotes, one of them is the question

C# — samples/L13.DebugLab/Batch.cs · #region BatchLoop

```
public static BatchResult Run(int count)
{
    var gross = 0m;
    var quoted = 0;
    var declined = new List<int>();
    for (var i = 0; i < count; i++)
    {
        var request = Request(i);
        try
        {
            // Break on this line, after request is assigned: "i == 137", or a hit count of 138.
            gross += QuoteCalculator.Price(request).Total.Amount;
            quoted++;
        }
        catch (QuoteDeclinedException)
        {
            declined.Add(i);
        }
    }
    return new BatchResult(quoted, gross, declined);
}
```

Aim, do not step. A plain breakpoint on the marked line — the first inside the `try`, after `request` is assigned — stops 240 times. A conditional one with `i == 137`, or a hit count of 138 (the loop starts at 0), stops once with `request` already holding quote 137; the same line as a tracepoint logging `{i}` and `{request.Coverage}` traces the whole run without stopping. A breakpoint on `var request = Request(i);` itself would stop before the assignment. The `try/catch` matters for 5.6: the four declined quotes are caught here, so a plain run never shows them.

5.5 · DebuggerDisplay, a type proxy and a browsable view on one result

C# — samples/L13.DebugLab/Batch.cs · #region BatchResult

```
// The debugger shows BatchResultView instead of the raw fields: the summary first, the
// declined indexes one click away. Nothing in the production API changes.
[DebuggerTypeProxy(typeof(BatchResultView))]
[DebuggerDisplay("{Quoted} quoted, {Declined.Count} declined")]
public sealed class BatchResult(int quoted, decimal gross, List<int> declined)
{
    public int Quoted { get; } = quoted;
    public decimal Gross { get; } = gross;
    public List<int> Declined { get; } = declined;
}

internal sealed class BatchResultView(BatchResult result)
{
    public string Summary => $"{result.Quoted} quoted, gross {result.Gross}";
    public int FirstDeclinedIndex => result.Declined.Count == 0 ? -1 : result.Declined[0];
    [DebuggerBrowsable(DebuggerBrowsableState.RootHidden)]
    public int[] DeclinedIndexes => [.. result.Declined];
}
```

Summary first, detail on demand. `[DebuggerDisplay]` gives the one-line value; `DebuggerTypeProxy` swaps the expansion for `BatchResultView`, whose `RootHidden` array lists the declined indexes directly. Keep the braces to fields and properties: Microsoft warns that a general expression is evaluated every time the value is shown, by the evaluator of the frame's language, and can change program state — its recommended pattern is a private `DebuggerDisplay` property (DebuggerDisplay).

5.6 · One exception setting finds the throw that a catch hides

C# — samples/L13.DebugLab/Tariff.cs · #region ClaimsLoading

```
// Set Debug > Windows > Exception Settings > "L13.DebugLab.QuoteDeclinedException" and
// the debugger stops on the throw line, even though the batch loop catches it.
public static decimal ClaimsLoading(int claims) => claims switch
{
    < 0 => throw new ArgumentOutOfRangeException(nameof(claims)),
    0 => 0m,
    1 => 0.10m,
    2 => 0.25m,
    _ => throw new QuoteDeclinedException("3+ claims in 5 years"),
};
```

Tick the type and the debugger stops at the `throw`, handled or not. Open Debug > Windows > Exception Settings, add `L13.DebugLab.QuoteDeclinedException` and run: execution stops on the `_ => throw` arm, on quote 47, before the loop's `catch` sees it — a first-chance exception. The window does not check spelling, and the setting is kept per solution (manage exceptions with the debugger).

5.7 · A window or a setting for each question

You ask	Open or set	What it does	Source
Which quote is the first to decline?	Exception Settings, tick the exception type	Breaks where it is thrown, even when a `catch` handles it	Exception docs
Why does F11 skip a method?	`[DebuggerStepThrough]` (on `Tariff.Satang`, in `Tariff.cs`, not printed); Just My Code	Steps over non-user code; the Call Stack collapses it to `[External Code]`	Just My Code docs
What does this library do?	Call Stack > Show External Code, then double-click a frame	Decompiles it (ILSpy-based, since 17.7) — C# only, and async code can come out incomplete	Decompile docs
Where is that library's source?	a library built with Source Link and its symbols	The debugger downloads the underlying source files as you step in; symbols can come from a symbol server such as NuGet.org's	Source Link docs
Which task is awaiting what?	Debug > Windows > Tasks, or the Tasks view of Parallel Stacks	Lists async/await tasks with status and parent — the tasks are those of lesson 05 (Async, Tasks & Concurrency); no sample here runs async code	Tasks window docs
How do I keep a value in view?	pin the data tip (the pushpin, Pin to source)	Stays on the source line as you step; can carry comments and, for .NET Core 3.0 and later, pinned members; Debug > Export DataTips saves them as XML	Data tips docs

5.8 · Hot Reload and Edit and Continue — the limits Microsoft documents

Kind	What the page lists	Source
Not available	Release or custom build configurations, trimming, ReadyToRun, F#, mixed-mode debugging and optimized code	Hot Reload docs
Not available (Edit and Continue page)	dump debugging, and attaching to a process unless `COMPLUS_ForceENC=1` is set	Supported code changes
Edits that need a restart, .NET 6+	modifying an interface; adding an abstract, virtual or override member; adding a destructor; changing a type parameter, base type or delegate type; deleting a type; an edit inside a catch or finally block that holds the active statement	Roslyn supported edits
Edits that no longer need one	adding a `using` (VS 16.10); adding or changing an attribute and renaming a parameter (17.0); deleting members other than fields and namespace declarations (17.3); renaming members other than fields and changing parameter or return types (17.4)	Roslyn supported edits
Visual Basic only	a member that contains `On Error` / `Resume`, or an `Aggregate`, `Group By`, simple `Join` or `Group Join` query clause, cannot be edited while the app runs — `QuoteCalculator.GrossTotal` in `L13.DebugLabVb` is one (not printed)	Roslyn supported edits

> **Which list to follow:** the older table on supported code changes still lists renames, attributes and `using` as edits that cannot be applied; for .NET 6+ and Visual Studio 2022 and later that page points to the Roslyn list above, which this table follows. • **Verified** 2026-09-21.

5.9 · Attaching to a process you did not start

Scenario	Process	Notes	Source
Local app started without the debugger	`appname.exe` · `iisexpress.exe`	Debug > Attach to Process, `Ctrl+Alt+P`; `Shift+Alt+P` reattaches	Attach docs
ASP.NET Core on IIS	`w3wp.exe` or `dotnet.exe`	in-process hosting uses `w3wp.exe` since .NET Core 3; the remote debugger must run on the server	Attach docs
A limited-user Visual Studio	processes in Session 0, such as `w3wp.exe`	not listed: run Visual Studio as an administrator, or use `vsjitdebugger.exe -p`	Attach docs

6 · Testing in the IDE

Test Explorer is a live view of the tests the CLI would run — group it, filter it, and debug one test with a click. Open it from Test > Test Explorer (`Ctrl+E, T`); the default hierarchy is Project, Namespace, Class, and Group By adds Duration, State and Target Framework. A `[Trait("Category", "Tariff")]` on an xUnit test appears under Group By > Traits and in the Traits filter, and the same string reaches the CLI: `dotnet test --filter "Category=Tariff"` ran 14 of this lesson's 18 tests (measured). Playlists save a hand-picked or rule-based list as a `.playlist` file you can commit (run unit tests with Test Explorer).

A `.runsettings` file is the test host's equivalent of a Gradle `tasks.test` block, and there are three ways to hand it over. Visual Studio picks up a file named exactly `.runsettings` at the solution root when Test > Configure Run Settings > Auto Detect runsettings Files is on (look at your own Test menu — a correctly placed file is ignored when it is off); the same menu selects another file; and the `RunSettingsFilePath` MSBuild property in a project or `Directory.Build.props` makes `dotnet test` read the same file without depending on that switch and overrides the solution-level choice (.runsettings files). The file below sets test-host parallelism, a timeout, fails an empty run and sets two environment variables on the test host — and a test proves the variables arrived. One trap: `TestRunParameters` are documented for MSTest's and NUnit's `TestContext` only — the page describes no xUnit route — so this xUnit project uses environment variables.

Coverage and Live Unit Testing are where the IDE and lesson 10's CLI gates differ. Visual Studio counts coverage in *blocks* — a sequence of code with one entry and one exit — with lines as an optional column, so its percentage is not the line or branch percentage lesson 10 (Testing, TDD & 100% Coverage) gates on, and a threshold must name which one it means. Test > Analyze Code Coverage for All Tests exports `.coverage`, XML or Cobertura (code coverage in Visual Studio). Live Unit Testing reruns the tests your edit can affect as you type; its page says Enterprise only (Live Unit Testing). And `xunit.v3.mtp-off` — the package lesson 10 (Testing, TDD & 100% Coverage) and this lesson's tests use — keeps the classic VSTest path instead of the Microsoft.Testing.Platform experience xUnit v3 projects get automatically (xUnit.net v3 and Microsoft.Testing.Platform).

6.1 · .runsettings — one file, two readers

MSBuild / XML samples/.runsettings

```
<?xml version="1.0" encoding="utf-8"?>
<!-- Named exactly ".runsettings", kept in the solution folder: Visual Studio finds it when Auto
     Detect runsettings Files is on; the test project's RunSettingsFilePath makes `dotnet test`
     read it either way. -->
<RunSettings>
  <RunConfiguration>
    <!-- 0 = one test host per core -->
    <MaxCpuCount>0</MaxCpuCount>
    <!-- an empty run is a failure, not a pass -->
    <TreatNoTestsAsError>true</TreatNoTestsAsError>
    <!-- a hung test host is killed after 60 seconds -->
    <TestSessionTimeout>60000</TestSessionTimeout>
    <EnvironmentVariables>
      <L13_RUNSETTINGS>applied</L13_RUNSETTINGS>
      <L13_BATCH_SIZE>240</L13_BATCH_SIZE>
    </EnvironmentVariables>
  </RunConfiguration>
</RunSettings>
```

Named exactly `.runsettings`, beside the solution. Visual Studio finds it by itself when Auto Detect runsettings Files is on; the test project's `RunSettingsFilePath` (next panel) makes `dotnet test` read the same file either way. `EnvironmentVariables` are set on the test host. Do not rely on a file in another folder being detected — only that exact name at the solution root is (.runsettings files).

6.2 · The test project — RunSettingsFilePath and the v3 packages

MSBuild / XML samples/L13.DebugLab.Tests/L13.DebugLab.Tests.csproj

```
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <IsPackable>false</IsPackable>
    <!-- dotnet test reads the same file Visual Studio finds beside the solution -->
    <RunSettingsFilePath>$(MSBuildThisFileDirectory)..\.runsettings</RunSettingsFilePath>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="18.10.0" />
    <PackageReference Include="xunit.v3.mtp-off" Version="4.0.1" />
    <PackageReference Include="xunit.runner.visualstudio" Version="4.0.0" PrivateAssets="all" />
  </ItemGroup>

  <ItemGroup>
    <ProjectReference Include="..\L13.DebugLab\L13.DebugLab.csproj" />
    <Using Include="Xunit" />
    <Using Include="L13.DebugLab" />
  </ItemGroup>

</Project>
```

One property connects the two worlds. `RunSettingsFilePath` points at `../.runsettings`; `Microsoft.NET.Test.Sdk` plus `xunit.v3.mtp-off` and `xunit.runner.visualstudio` keep the classic Test Explorer and `dotnet test` path, the same set lesson 10 (Testing, TDD & 100% Coverage) uses.

6.3 · The same idea: test-run environment, in Gradle and in .runsettings

Kotlin — for comparison `build.gradle.kts + a JUnit test`

```kotlin
// build.gradle.kts
tasks.test {
    environment("L13_RUNSETTINGS", "applied")
    environment("L13_BATCH_SIZE", "240")
    maxParallelForks =
        Runtime.getRuntime().availableProcessors()
}

// RunConfigTest.kt
@Test
fun `run configuration was applied`() {
    assertEquals("applied",
        System.getenv("L13_RUNSETTINGS"))
    assertEquals("240",
        System.getenv("L13_BATCH_SIZE"))
}
```

C# `RunSettingsTests.cs · TestsRunSettings`

```csharp
// Passes only if the .runsettings file was applied.
[Fact, Trait("Category", "Environment")]
public void The_runsettings_file_was_applied()
{
    Assert.Equal("applied", Env("L13_RUNSETTINGS"));
    Assert.Equal("240", Env("L13_BATCH_SIZE"));
}

private static string? Env(string name) =>
    Environment.GetEnvironmentVariable(name);
```

Both hand the test process its environment before the first test runs — and both need a test that proves it. Gradle keeps that in the build script; .NET keeps it in a file both Visual Studio and the CLI read. Without the assertion, a silently ignored settings file looks like a passing run.

6.4 · A trait, and the worked example as a test

C# `samples/L13.DebugLab.Tests/TariffTests.cs · #region TestsWorkedExample`

```csharp
// Test Explorer groups by trait: Trait > Category > Tariff. The CLI filter is the same string.
[Fact, Trait("Category", "Tariff")]
public void Worked_example_prices_to_the_curriculum_total()
{
    var request = new QuoteRequest(
        new Vehicle("Make0", "Model0", 2024, 1500, VehicleUse.Private, Money.Thb(550_000)),
        new Driver(new DateOnly(2002, 6, 15), 4, 0), CoverageClass.Class1, QuoteBatch.StartDate);

    var premium = QuoteCalculator.Price(request);

    Assert.Equal(8316.00m, premium.Net.Amount);
    Assert.Equal(33.26m, premium.StampDuty.Amount);
    Assert.Equal(584.45m, premium.Vat.Amount);
    Assert.Equal(8933.71m, premium.Total.Amount);
}
```

The trait is the Test Explorer group and the CLI filter at once. The expected figures are the track's worked example (illustrative tariff, not a real one): Class 1, 550,000 THB, private use, a 23-year-old with 4 claim-free years. The program prints `worked example: net 8316.00 · stamp duty 33.26 · VAT 584.45 · total 8933.71 THB` and the test asserts the same four numbers.

6.5 · Test Explorer, the CLI and lesson 10's gates — the same tests, three doors

In Visual Studio	On the command line	Watch for
Group By > Traits, or the Traits filter	`dotnet test --filter "Category=Tariff"`	Filter syntax is the framework's — the string is the same, the quoting differs by shell
Run All (`Ctrl+R, V`)	`dotnet test`	Both must give 18 — a difference means settings or SDK differ (1.2)
Test > Configure Run Settings	the `RunSettingsFilePath` property	Autodetect needs the exact file name at the solution root and its option on
Analyze Code Coverage for All Tests	coverlet or Microsoft code coverage, as in lesson 10 (Testing, TDD & 100% Coverage)	Blocks in the IDE, line and branch in lesson 10 — never compare the percentages
Live Unit Testing	no equivalent used in this track	Enterprise only on its own Learn page; the page for your version decides

Traits partition the suite: 14 + 3 + 1 = 18. The single Environment test fails if the settings file is not applied — the cheapest guard against a `.runsettings` that quietly stopped being read. • **Measured** by `dotnet test --filter Category=....`

7 · Code quality and AI assistance in the IDE

A rule lives in an `.editorconfig` line, and the editor and the compiler read the same line. The severity decides what you see: *suggestion* is a quiet marker, *warning* and *error* are squiggles. Whether a squiggle also fails `dotnet build` depends on the

rule: compiler warnings and analyzer packages always run at build, but an `IDE`-prefixed style rule is evaluated by the build only when `EnforceCodeStyleInBuild` is true (code analysis overview — enforce code style on build); with the repository's `TreatWarningsAsErrors` the warning then fails it for everyone. The Options dialog is the trap: code-style settings there belong to your Visual Studio account, are not associated with a codebase and are not enforced at build or in CI — and a setting in an `.editorconfig` takes precedence over them (code style options and Code Cleanup). Analyzers, severities and `dotnet format` belong to lesson 07 (Solutions, MSBuild & Mono-repos); this is what they look like from the editor.

Code Cleanup applies the rules you already chose, on demand or on save. `Ctrl+K, Ctrl+E` runs a configured cleanup profile — remove unused usings, apply `var` and brace preferences and more. A severity of *Refactoring Only* keeps a rule out of cleanup and leaves it as a Quick Action, and Options > Text Editor > Code Cleanup has a switch to run a profile on save. For the whole solution the CLI's `dotnet format` does the same job, as lesson 07 (Solutions, MSBuild & Mono-repos) shows — pin `end_of_line` first, or Windows and Linux disagree about what is clean.

GitHub Copilot is part of the IDE workflow now, and Microsoft's pages describe it by GitHub plan, not by Visual Studio edition. On the Visual Studio 2026 page, the Copilot badge opens the chat, and someone without a subscription can start on Copilot Free. Chat has Ask, Agent and Plan modes; built-in agents cover debugging, profiling, testing and modernization; and repository files such as `.github/copilot-instructions.md` carry your team's instructions — a file you review like code (Get started with GitHub Copilot in Visual Studio · GitHub Copilot Free in Visual Studio). **Review what it writes as you would a colleague's pull request** — the gates in lesson 10 (Testing, TDD & 100% Coverage) and lesson 07 (Solutions, MSBuild & Mono-repos) apply to generated code exactly as to typed code.

7.1 · .editorconfig — the same severity in the editor and in the build

Text — `samples/.editorconfig`

```
# Read by Visual Studio while you type AND by the compiler: the same rule, the same severity.
# root = true stops the search, so a stray .editorconfig higher up cannot change these rules.
root = true

[*]
end_of_line = lf
insert_final_newline = true

[*.{cs,vb}]
indent_style = space
indent_size = 4

[*.cs]
# always a squiggle in the editor; a build error only because Directory.Build.props turns on
# EnforceCodeStyleInBuild and the repository's TreatWarningsAsErrors turns the warning into an error
csharp_style_namespace_declarations = file_scoped:warning
# a suggestion shows as three dots under the code and never fails the build at all
csharp_style_prefer_primary_constructors = true:suggestion

# tests read as sentences: allow underscores in test method names
[L13.DebugLab.Tests/**.cs]
dotnet_diagnostic.CA1707.severity = none
```

Two rules, two severities. `file_scoped:warning` is a squiggle in the editor, and a build error only because of 7.3; the primary-constructor line is a `suggestion` — a marker with a Quick Action that never fails anything. `root = true` stops the search, so an `.editorconfig` higher up cannot change these rules.

7.2 · Where a style setting can live, and who sees it

Place	Seen by	Enforced at build	Travels with the repository
Tools > Options > Text Editor > Code Style	you	no	no
`.editorconfig`, an `IDE` style rule at warning or error	everyone who opens the code	only with `EnforceCodeStyleInBuild` (on in 7.3)	yes
`.editorconfig` with *suggestion* or *Refactoring Only*	everyone in the editor	no — it is a hint or a Quick Action	yes
A compiler warning, or a Roslyn analyzer package as in lesson 07 (Solutions, MSBuild & Mono-repos)	everyone	yes, at the severity you set	yes

Source: code style options and Code Cleanup · code analysis overview — enforce code style on build. Since Visual Studio 2019 16.8 the .NET code-style rules can be enforced at build, and a handful of them still apply only in the IDE. • Verified 2026-09-21.

7.3 · Directory.Build.props — the switch that turns a style squiggle into a build error

MSBuild / XML samples/Directory.Build.props

```
<Project>
  <!-- MSBuild imports only the NEAREST Directory.Build.props: chain to the parent by hand -->
  <Import Project="$([MSBuild]::GetPathOfFileAbove('Directory.Build.props',
                                                   '$(MSBuildThisFileDirectory)../'))" />

  <PropertyGroup>
    <!-- IDE-style rules from .editorconfig then fail dotnet build too, not only the editor -->
    <EnforceCodeStyleInBuild>true</EnforceCodeStyleInBuild>
  </PropertyGroup>
</Project>
```

Chain to the parent, then flip one property. MSBuild imports only the nearest `Directory.Build.props`, so this file imports the repository's by hand, as in lesson 07 (Solutions, MSBuild & Mono-repos) §4.2, and adds `EnforceCodeStyleInBuild`. • **Measured** by `capture.py`: a block-scoped namespace fails `dotnet build` with **IDE0161**; with the property off (`-p:EnforceCodeStyleInBuild=false`) the same file builds.

8 · Performance and diagnostics

Visual Studio has two performance tools, and the difference is whether you are already debugging. The Diagnostic Tools window (`Ctrl+Alt+F2`) opens during a debug session with CPU, memory and .NET counter graphs and PerfTips on the source lines. The Performance Profiler (Debug > Performance Profiler, `Alt+F2`) is aimed at Release builds and offers tools for .NET — CPU Usage, .NET Object Allocation, Memory Usage, Instrumentation (exact call counts), .NET Async, File I/O, .NET Counters, Database and GPU; in Visual Studio 2026 they are grouped into Flexible and Standalone tabs, with a Copilot "Recommended" tab in front. IntelliTrace events in the Diagnostic Tools are an Enterprise feature (the profiling tools overview).

Every tool has a command-line sibling, and you will need it on a server without an IDE. `dotnet-counters`, `dotnet-trace`, `dotnet-dump`, `dotnet-stack` and `dotnet-monitor` are Microsoft's cross-platform diagnostics tools (.NET diagnostics tools); Visual Studio's profiler can open a trace that `dotnet-trace` collected, which is the workflow for a problem that only shows in production. You know the shape from the JVM: a JFR recording collected on the server, opened in JDK Mission Control on your desk.

This lesson prints only figures a script can reproduce: allocated bytes and call counts, never wall-clock time. `L13.PerfLab` builds one report three ways and reads `GC.GetAllocatedBytesForCurrentThread` around each; the same code on the same runtime and configuration allocates the same number of bytes, so the Release table below is identical on every run (the capture script asserts it). The configuration matters: a Debug build of the same program prints other absolute figures (floor 107,928, LINQ + string.Format 279,328 bytes), so every figure here names `-c Release`. A timing from this box would differ on yours, so none is printed — use the profiler for that, on your own hardware.

8.1 · Measuring bytes, not time

C# samples/L13.PerfLab/Program.cs · #region Measure

```
// GC.GetAllocatedBytesForCurrentThread counts bytes, not time, so the figure is repeatable:
// the same code on the same runtime allocates the same amount on every run.
static (string Text, int Calls, long Bytes) Measure(
    Func<IReadOnlyList<QuoteRequest>, string> version, IReadOnlyList<QuoteRequest> input)
{
    version(input);                                  // warm-up: static setup and first-call costs
    Report.PriceCalls = 0;
    var before = GC.GetAllocatedBytesForCurrentThread();
    var text = version(input);
    var bytes = GC.GetAllocatedBytesForCurrentThread() - before;
    return (text, Report.PriceCalls, bytes);
}
```

Warm up, count, run once, subtract. The first call pays static-setup costs, so it is thrown away and the counter is reset after it; `Report.PriceCalls` therefore counts one measured pass, which is not what a profiler over the whole run would total; the delta of `GetAllocatedBytesForCurrentThread` is the report's allocation. Figures: `dotnet run -c Release`.

8.2 · The allocation-heavy version

C# — samples/L13.PerfLab/Report.cs · #region HeavyVersion

```
// Allocates on every line: a closure, boxed arguments for string.Format, a formatted string per
// line, a List<string>, and the Join. It also prices every quote twice.
public static string Heavy(IReadOnlyList<QuoteRequest> requests) =>
    string.Join("\n", requests
        .Select((r, i) => string.Format("{0},{1},{2},{3}", i, r.Coverage,
            Price(r).Net.Amount, Price(r).Total.Amount))
        .ToList());
```

One LINQ expression, four kinds of garbage — and two `Price` calls per row. The `Select` lambda is a closure, `string.Format` boxes its four arguments and builds a string per line, `ToList` keeps them all, and `Join` copies them again. Both versions return the same text, byte for byte; the program checks it and exits non-zero if not.

8.3 · The allocation-light version

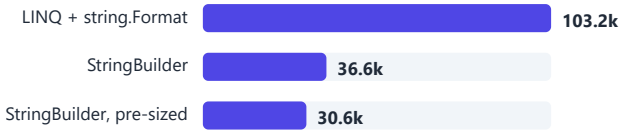
C# — samples/L13.PerfLab/Report.cs · #region PresizedVersion

```
private static readonly string[] CoverageNames = Enum.GetNames<CoverageClass>();

// Prices once, reuses the enum names, and reserves the builder's capacity up front.
public static string Presized(IReadOnlyList<QuoteRequest> requests)
{
    var sb = new StringBuilder(requests.Count * 36);
    for (var i = 0; i < requests.Count; i++)
    {
        var p = Price(requests[i]);
        if (i > 0) sb.Append('\n');
        sb.Append(i).Append(',').Append(CoverageNames[(int)requests[i].Coverage]).Append(',')
          .Append(p.Net.Amount).Append(',').Append(p.Total.Amount);
    }
    return sb.ToString();
}
```

Price once, reuse the enum names, reserve the buffer. Compare with 8.2: no closure, no boxing, no per-line string, one `StringBuilder` sized up front, one `Price` call per quote.

8.4 · Bytes the report allocates above pricing alone (Release)



Release build · allocated bytes minus the 39,680-byte pricing floor · the same 6,794-character report each time · measured

In a Release build the heavy report allocates 103,152 bytes above the floor; the pre-sized one 30,648 — 3.4 times less. • Measured by `samples/tools/capture.py`; a Debug build gives other figures, so the ratio is tied to Release.

8.5 · Calls to Price in one measured pass



the sample's own counter, one measured pass (reset after the warm-up) · 236 quotes are priced

The heavy version calls `Price` twice per quote — 472 calls for 236 quotes. Some of its extra allocation is duplicated work, not formatting, which a bytes table alone would hide. • Measured The profiler was not run: over the whole program its totals would include the warm-up pass — 472 for a single-pass version, 944 for `Report.Heavy` • Estimate.

9 · Legacy estates in the IDE

The IDE is where a legacy estate stops being abstract: you open the old solution, build it with the MSBuild you already know, and debug what runs. Two facts from Microsoft's pages explain the parts that surprise people. The WinForms designer for .NET runs out of process, in a `DesignToolsServer.exe` beside `devenv.exe` on the same .NET version and platform as your app — Visual Studio itself is a .NET Framework application, so the designer for a modern .NET form has to live outside it (the WinForms designer for .NET). And for a site already running on IIS you do not run it from the IDE: you attach to `w3wp.exe`, with Visual Studio elevated so the worker process is listed (attach to running processes).

What the IDE does not do here is decide how to migrate. Lesson 06 (VB.NET for Legacy Estates) teaches the VB and WinForms code you will meet and lesson 12 (Cloud-Native .NET on AWS & Modernization) owns the modernization tooling; this lesson only notes that they all run inside the same solution, the same MSBuild and the same debugger. Two limits are

worth knowing before you promise anything: decompilation generates C# only — never Visual Basic — and can be incomplete for async code (decompile .NET code while debugging). The WPF designer is **not** covered: this lesson verified the WinForms designer page only, so read the WPF one before you rely on it.

Not covered here, and why: IIS Express versus a full IIS site and the `web.config` settings a legacy site adds are not compared — this lesson read only the attach-to-process page, which does not compare them; a preview Visual Studio channel appears only as the Insiders channel named in 2.2.

10 · Other editors — VS Code with the C# Dev Kit, and Rider

Two other tools are worth knowing, and the licence condition is what most often decides between them. The C# Dev Kit is Microsoft's extension for VS Code: free for personal, academic and open-source projects, and free for commercial use by teams of up to five; for six or more, its FAQ says a Visual Studio Professional or higher subscription is needed, and the get-started page says you sign in to a Visual Studio subscription to use it. The source of the base C# extension is MIT-licensed; read its terms for the shipped build (C# Dev Kit licensing FAQ · C# in VS Code). JetBrains Rider is free for non-commercial use; JetBrains publishes the commercial terms (JetBrains Rider). Read the page on the day you buy — terms change.

For a JVM developer the honest choice is about habits and estate, not features. The right-hand column below is the author's judgement, labelled as such: it is not a benchmark and no vendor page says it.

10.1 · A decision table — vendor facts verified, the last column is an ESTIMATE

Tool	What the vendor page states (verified 2026-09-21)	Source	Choose it when (judgement)
Visual Studio Community	the full IDE, free for individuals, classroom, academic, open source and non-enterprise organisations up to 5 users	Editions page	you qualify and want the full debugger and designers
Visual Studio Professional	the paid team edition; LTSC servicing channels	Editions page	the team works on legacy WinForms/IIS/VB estates or needs LTSC servicing
VS Code + C# Dev Kit	free for personal, academic, open source, and commercial teams of up to 5; larger commercial teams need a Visual Studio Professional or higher subscription; creates and runs projects with `.NET: New Project`; sign-in to a Visual Studio subscription required	Dev Kit FAQ · C# in VS Code	you work in TypeScript and Python too and want one light editor for services
JetBrains Rider	free for non-commercial use; a cross-platform .NET IDE whose page lists inspections, debugging, unit testing, NuGet and version control, and ASP.NET, Blazor, MAUI, WPF and WinForms among its technologies	Rider page	you want the IntelliJ family's habits and your licence terms allow it

11 · Hands-on — a guided debug session over a correct program

The best way to learn a debugger is to ask it questions whose answers you can check. `L13.DebugLab` prices 240 generated quotes with the track's illustrative tariff; four are declined, and every test passes. Nothing is broken: each question below is answered by one IDE feature, and the CLI column reaches the same answer without an IDE, so you can tell whether the feature or your reading is at fault. The IDE steps below were not run — there is no Visual Studio on the build machine (• Estimate) — while the CLI column and its answers were run.

1. Open `samples/L13.slnx` in Visual Studio; the `.vsconfig` offers any missing workload.
2. Make `L13.DebugLab` the startup project and pick the profile named in the row from the run dropdown — `Batch` for questions 1–4. The two Show profiles skip the batch loop, so a breakpoint in `QuoteBatch.Run` never hits there.
3. Set the exception setting or breakpoint the row names, press F5, and read the value where the row says.
4. Compare it with the answer. If they differ, run the CLI column: its output is the reference.

11.1 · Five questions, the IDE feature that answers each, and the answer

Question	In Visual Studio	Without it	The answer (measured)
1 · Which quote declines first?	profile `Batch` · Exception Settings: tick `QuoteDeclinedException`, F5; double-click the `QuoteBatch.Run` frame in the Call Stack, read `i` in Locals	`dotnet run … -- --show 47`	index 47 — the four are 47, 108, 169, 230

Question	In Visual Studio	Without it	The answer (measured)
2 · What does quote 137 price to?	profile `Batch` · a conditional breakpoint `i == 137` (or hit count 138) on the marked line of 5.4; read `request` in a data tip	`dotnet run … -lp "Show quote 137"`	Class2Plus, sum insured 640000.00, loadings 10%, no-claim 0% → net 8448.00 · total 9075.52 THB
3 · Where is that exception thrown?	profile `Batch` · the same Exception Settings stop; read the top frame of the Call Stack	the message names the throwing method	`Tariff.ClaimsLoading`, the `_ => throw` arm
4 · What is the total after the loop?	profile `Batch` · a breakpoint on the line after `QuoteBatch.Run` in `Program.cs`; read `Summary` in the type proxy view of `result`	`dotnet run …`	1458887.59 THB
5 · Which report prices twice?	• **Estimate** Performance Profiler > Instrumentation on `L13.PerfLab`, a Release build	`dotnet run -c Release --project …L13.PerfLab`	`LINQ + string.Format` — 472 calls for 236 quotes (Release run)

11.2 · The same answers from the command line

Terminal — captured from a real run — output of samples/tools/capture.py · path line omitted · header lines shown once

```
$ dotnet run --project samples/L13.DebugLab -lp "Show quote 137"
MotorQuote DebugLab — illustrative tariff, not a real one
worked example: net 8316.00 · stamp duty 33.26 · VAT 584.45 · total 8933.71 THB
quote #137: Class2Plus, sum insured 640000.00, loadings 10%, no-claim 0%
quote #137: net 8448.00 · total 9075.52 THB

$ dotnet run --project samples/L13.DebugLab
batch of 240 quotes: 236 quoted, 4 declined
declined at index: 47, 108, 169, 230
gross premium of the quoted policies: 1458887.59 THB

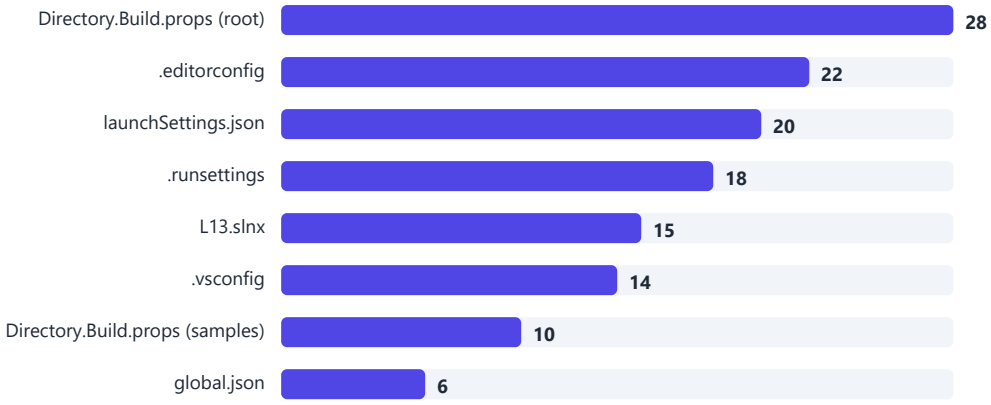
$ dotnet run --project samples/L13.DebugLab -- --show 47
quote #47: declined in ClaimsLoading — 3+ claims in 5 years
```

Both routes must agree. `dotnet run -lp` prints one extra first line naming the launch-settings file it used, and every run prints the same two header lines; both are omitted after the first run here. The figures are the tariff's own: the worked example prices to 8,933.71 THB — net 8,316.00, stamp duty 33.26, VAT 584.45 — and the four declined indexes are the ones the tests assert. • **Measured**.

11.3 · The files Visual Studio reads in this lesson, and who else reads them

File	Read by	Controls	Section
`L13.slnx`	Visual Studio 17.14+, the .NET CLI	the project list and solution folders	4.2
`.vsconfig`	the Installer, and Visual Studio when the solution opens	which components are installed	3.1
`global.json`	the `dotnet` muxer and MSBuild's SDK resolver	which SDK builds, per lesson 01 (The .NET Platform Map)	3
`Directory.Build.props`	MSBuild — IDE and CLI alike	target framework, nullable, warnings as errors, and in the samples `EnforceCodeStyleInBuild`; see lesson 07 (Solutions, MSBuild & Mono-repos)	7.3
`.editorconfig`	the editor, the compiler, `dotnet format`	style and analyzer severities	7.1
`.runsettings`	Test Explorer (with Auto Detect on); `dotnet test` via a property	test-host environment and limits	6.1
`launchSettings.json`	the F5 dropdown, `dotnet run -lp`	start-up profiles	5.1
`Directory.Build.rsp`, `Directory.Solution.props` / `.targets`	the command line only: the response file, and MSBuild's solution build (MSBuild response files; lesson 07 (Solutions, MSBuild & Mono-repos) §4.5)	default switches and solution-level steps — the IDE ignores them	1.2

11.4 · Every file that configures the IDE or the build, by size



lines per file · the samples' own settings files and the repository root's global.json and Directory.Build.props · measured from disk at build time

8 small files carry the whole setup; none is longer than 28 lines — short enough to live in the repository, where a colleague and CI read it too. • **Measured** from disk by `capture.py`.

11.5 · What to carry over, learn fresh and unlearn

Carries over unchanged

- Run configurations as named, shareable profiles
- Conditions and actions on breakpoints
- A rule file that travels with the repository
- Attach to a process — and match its symbols
- Selecting tests by tag from the IDE and the CLI

Learn fresh

- Exception Settings and the first-chance stop
- `[DebuggerDisplay]` and a type proxy
- `.runsettings` and `RunSettingsFilePath`
- Just My Code and `[External Code]`
- `.vsconfig` in the solution root

Unlearn

- "The Options dialog holds team settings"
- "Coverage needs Enterprise" — check the page for your version
- "Hot Reload works in Release builds"
- "The IDE builds differently from the CLI" — only a workload, an SDK or a command-line-only file does
- "A profiler timing from my laptop is a benchmark"

⚠ Traps

1. **The Options dialog looks like team configuration and is not.** Code-style settings there belong to your Visual Studio account and are not enforced at build or in CI; an `.editorconfig` is the file that travels, and it wins over the dialog — but an `IDE` style rule in it fails `dotnet build` only when `EnforceCodeStyleInBuild` is on (code style options and Code Cleanup).
2. **"Coverage needs Enterprise" is out of date — and two Microsoft pages still disagree.** The coverage page says Enterprise, Community and Professional in Visual Studio 2026; the Test Explorer page still says Enterprise in one sentence; Live Unit Testing, Fakes and IntelliTrace events are still Enterprise (code coverage in Visual Studio · run unit tests with Test Explorer · Live Unit Testing). Read the page for your version, and confirm in your own Test menu.
3. **A `.runsettings` is autodetected only under that exact name at the solution root, and only while Auto Detect runsettings Files is on** — and `TestRunParameters` are documented only for MSTest and NUnit, so an xUnit project reads environment variables instead (.runsettings files).
4. **Hot Reload is a debug-session feature with a published list of limits — and the list is shorter than older pages say.** Release and optimized builds, attaching without `COMPLUS_ForceENC=1`, edits to interfaces, new abstract, virtual or override members, destructors, deleted types and, in Visual Basic, members with `Aggregate` or `On Error` end in a restart; renames, signature changes, attributes and a new `using` are supported in current Visual Studio, although the older table on the supported-code-changes page still lists them (Hot Reload · Supported Edits in Edit and Continue and Hot Reload).
5. **Upgrading Visual Studio removes the .NET SDK it installed earlier** — even a different major — and a stand-alone SDK you install to avoid that is no longer updated by Visual Studio (global.json and rollForward). Decompiled code is never the original: C# only, and async can be incomplete (decompile .NET code while debugging).

✓ Checkpoint — you can work in Visual Studio when

1. `python 0-script/verify_samples.py --only 13` reports 4/4 passed, and `samples/tools/capture.py` reproduces the 18 tests, the four declined indexes and the Release allocation table on your machine.
2. You can explain why Build-in-Visual-Studio and `dotnet build` agree, and name what legitimately makes them differ: a workload, an SDK, and the two command-line-only files (`Directory.Build.rsp`, `Directory.Solution.props` / `.targets`).
3. You can pick an edition from the published licence rules and say which gated features you checked on which page — and which page you would trust if two disagree.
4. Given a loop that catches an exception, you can find the first throw and the 137th iteration without adding a line of code, and reach the same answers from the CLI.

🔍 Check yourself

1. A build passes in Visual Studio but `dotnet build` says "No .NET SDKs were found". Which environment variable do you check first, and where does the IDE's own SDK come from?
2. Which kind of organisation is excluded from Community, and which features does this lesson list as still Enterprise-only? Which page do you trust when two Microsoft pages disagree?
3. `dotnet test --filter "Category=Tariff"` ran 14 tests. Which Test Explorer control selects the same set, and which attribute makes both work?
4. Give two breakpoint settings that stop on quote 137 of 240 with no code change, and the one setting that stops on each of the four declined quotes.
5. Why does the pre-sized report allocate less than the heavy one, and why would a profiler over the whole program total more `Price` calls than the sample's own counter?
6. Your `.runsettings` is in a folder called `settings` and Test Explorer ignores it. Why — and which property makes `dotnet test` read it anyway?
7. A colleague edits a Visual Basic member that contains `Aggregate` while debugging and Hot Reload refuses. What do you do, and which page lists the limit?

Lesson 13 in one line: Visual Studio is a front end to files you commit — `.slnx`, `.vsconfig`, `.editorconfig`, `.runsettings`, `launchSettings.json` — so the IDE and the CLI reach the same build and the same tests; its own strengths are the debugger you can aim, Test Explorer and the profiler, and every claim about editions and gating is one you check on Microsoft's page the day you decide (read 2026-09-21).
Back to the track: lesson 07 (Solutions, MSBuild & Mono-repos) for the files, lesson 10 (Testing, TDD & 100% Coverage) for the gates these tools reach, and the official Visual Studio documentation for everything newer than this PDF.